

SAINTGITS COLLEGE OF ENGINEERING

(Autonomous), Kottayam

B.TECH MINOR PROJECT REPORT

entitled

KINETIQ

Real-Time Wellness Mirror with FreeRTOS and Sensor-Grounded AI

Submitted by

Sidharth Sajith and Tharun Oommen Jacob

Under the guidance of

Er. Jittu George Eapen

Assistant Professor

Department of Electronics Engineering

Saintgits College of Engineering (Autonomous), Kottayam

Academic year: *[to be entered by the department]*

CERTIFICATE

This is to certify that the B.Tech Minor Project entitled **"KINETIQ – Real-Time Wellness Mirror with FreeRTOS and Sensor-Grounded AI"** is a record of work carried out by **Sidharth Sajith** and **Tharun Oommen Jacob** in the Department of Electronics Engineering, Saintgits College of Engineering (Autonomous), Kottayam, under the guidance of **Er. Jittu George Eapen**, Assistant Professor. The institutional approvals, register numbers, submission date, and signatures shall be completed by the authorised office.

Supervisor Er. Jittu George Eapen	Head of Department *[Authorised signatory]*
Project Coordinator *[Authorised signatory]*	Principal *[Authorised signatory]*

Place: Kottayam Date: *[to be entered]*

ACKNOWLEDGEMENTS

The authors record sincere gratitude to Er. Jittu George Eapen, Assistant Professor, for technical guidance and review. Appreciation is extended to the Department of Electronics Engineering and Saintgits College of Engineering (Autonomous) for the academic environment supporting the work. The documented implementation also benefited from the project team's test materials, device records, and software artefacts.

ABSTRACT

KINETIQ is a local wellness-mirror prototype that combines two deliberately distinct telemetry pathways with a common validation and assistance layer. The genuine-data pathway transports observations from a Huawei Band 6 through Huawei Health and an Android accessibility streaming service to an authenticated local receiver. The receiver and bridge admit only complete, fresh, non-simulated values before forwarding a constrained mirror payload. A separate native FreeRTOS v11 application demonstrates deterministic activity-control behaviour with synthetic telemetry, queue and timing diagnostics, and an operator-controlled fallback. Both pathways terminate at a laptop-based mirror that exposes source, link, freshness, and data-quality state instead of presenting cached values as live.

The shared contract contains eight fields: timestamp, heart rate, oxygen saturation, activity, motion intensity, exertion, steps, and PPG quality. Schema, range, completeness, source, and freshness checks gate dashboard and assistant inputs. The assistant receives accepted context, returns a bounded wellness-oriented response, displays a grounding verdict, and retains an offline deterministic template when the optional local language-model service is unavailable. The implementation is explicitly non-clinical: it neither diagnoses nor treats a condition.

Repository evidence includes authenticated receiver records containing 60 BPM, 100% SpO_2, stress 57, and 4,181 steps on 22 October 2026; a separate synthetic dashboard session containing 75 frames over 15 seconds with no recorded drops; a 30-message BLE mock; and a 0.48 s faster-whisper result on synthesized audio. These observations establish bounded portions of the pipeline only. Physical ESP32 sensing remains represented by component selection and Wokwi simulation rather than a verified final hardware acquisition.

Keywords: FreeRTOS, wearable telemetry, data freshness, wellness mirror, schema validation, sensor-grounded AI, Huawei Health.

Contents

Acknowledgements ii

Abstract iii

List of Figures vi

List of Tables vii

Abbreviations viii

1 INTRODUCTION 1

1.1 Background and Motivation 1
1.2 Engineering Problem 1
1.3 Limitations of Unqualified Wellness Interfaces 2
1.4 Objectives 2
1.5 Scope and Boundaries 3
1.6 Dual-Pathway Overview 3
1.7 Chapter Organisation 3

2 LITERATURE REVIEW AND TECHNICAL BACKGROUND 4

2.1 Real-Time Tasks and Embedded Telemetry 4
2.2 Wearable Data and Android-Mediated Integration 4
2.3 Validation, Freshness, and Provenance 5
2.4 Local Transport and Interface Security 5
2.5 Speech and Context-Grounded Assistance 5
2.6 Embedded Hardware Context 6
2.7 Engineering Gap 6

3 OBJECTIVES AND SYSTEM REQUIREMENTS 7

3.1 Requirement Basis 7
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 9
3.4 Shared Telemetry Contract 9
3.5 Safety and Acceptance Boundary 10
3.6 Demonstration Acceptance Conditions 10

4 SYSTEM DESIGN AND ARCHITECTURE 11
4.1 Three-Tier Architecture 11
4.2 Genuine Wearable-Data Pathway 11
4.3 Deterministic FreeRTOS Control Pathway 12
4.4 Telemetry Validation and Freshness Workflow 13
4.5 Dashboard and Assistance Layer 13
4.6 Hardware Design Scope 14

5 IMPLEMENTATION AND METHODOLOGY 16
5.1 Software Organisation 16
5.2 FreeRTOS Tasks, Queue, and Synchronisation 17
5.3 Genuine-Path Ingestion and Bridging 17
5.4 Schema Validation and Dashboard State 18
5.5 Grounded Assistance and Voice Route 19
5.6 Launch and Fallback Procedure 19
5.7 Hardware Simulation and Integration Status 20

6 RESULTS AND DISCUSSION 21
6.1 Evidence Classification 21
6.2 Genuine Wearable-Path Observation 21
6.3 Schema and Bridge Validation 22
6.4 Dashboard Mock Stream 22
6.5 BLE Mock and Voice Observation 24
6.6 RTOS Control and Offline Fallback 24
6.7 Grounding Verdict Behaviour 24
6.8 Hardware and Budget Reconciliation 25
6.9 Overall Verification Summary 25

7 CONCLUSION AND FUTURE SCOPE 26
7.1 Conclusion 26
7.2 Future Scope 26

A SUPPORTING TECHNICAL INFORMATION 28
A.1 Operator Command Map 28
A.2 Selected Receiver Metadata 28

REFERENCES 30

List of Figures

1.1 Two telemetry pathways with a shared mirror interface 3

4.1 Overall KINETIQ architecture 11
4.2 Validation and freshness workflow 13
4.3 Dashboard mirror with synthetic stream (not live) 14
4.4 Earlier hardware-path conceptual architecture 15

5.1 Native FreeRTOS control data path 17

6.1 Dashboard freshness states (synthetic input) 23
6.2 Mock telemetry session (synthetic, 30 samples) 23

List of Tables

3.1 Functional requirements and available verification evidence. 8
3.2 Non-functional requirements and design controls. 9
3.3 Eight-field telemetry contract enforced by the shared receiver. 10

4.1 Native FreeRTOS task structure in the examined implementation. 12

5.1 Principal implementation modules and responsibilities. 16
5.2 Dashboard freshness and link-state treatment. 19

6.1 Evidence classification used for reported observations. 21
6.2 Selected documented Huawei-Health relay observations at the local receiver. 22
6.3 Reconciled component subtotal from the displayed final-review BOM. . . . 25

A.1 Native FreeRTOS control commands. 28

ABBREVIATIONS

AI	Artificial Intelligence
BLE	Bluetooth Low Energy
BOM	Bill of Materials
CSV	Comma-Separated Values
ESP32	Espressif 32-bit microcontroller family
HWM	High-Water Mark
IMU	Inertial Measurement Unit
JSON	JavaScript Object Notation
LLM	Large Language Model
PPG	Photoplethysmography
RTOS	Real-Time Operating System
SpO_2	Peripheral oxygen saturation
STT	Speech-to-Text
TTS	Text-to-Speech
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Wearable devices can make physiological observations available continuously, but a value on a wearable application screen is not automatically a current, validated value at a second interface. Transport delay, incomplete packets, cached application screens, and loss of connectivity can all separate the observation from its later presentation. A wellness interface that obscures those conditions risks making stale values appear live. KINETIQ addresses that interface and integration problem with explicit provenance, freshness, and validation states.

The implementation joins a Huawei Band 6 and Huawei Health data path to a laptop-based wellness mirror through a local Android accessibility streaming service and authenticated receiver. Android accessibility services are intended to observe and assist interaction with user-interface content under a user-granted permission model [1]. No direct Huawei Band application-programming-interface integration is claimed: the genuine pathway depends on data rendered by Huawei Health and captured by the phone-side service. The distinction matters because a visible Huawei Health value, a receiver record, an accepted bridge payload, and a rendered mirror value represent different stages of evidence.

KINETIQ also contains a separate native FreeRTOS v11 control pathway. Its virtual activity states and generated telemetry provide a deterministic, inspectable control demonstration when wearable transport is unavailable. FreeRTOS queues provide safe task-to-task data transfer, while mutexes protect a shared resource from concurrent access [2, 3]. The pathway is intentionally synthetic; its values are never presented as a person's physiological measurements.

1.2 Engineering Problem

Generic conversational assistance lacks direct access to the state that a user sees on a wellness dashboard. Conversely, passing arbitrary or stale telemetry to an assistant can make a fluent answer appear more reliable than its input. The engineering problem is therefore to combine local telemetry processing and wellness assistance while retaining five properties:

- a traceable distinction between genuine wearable observations and synthetic control telemetry;
- rejection of malformed, incomplete, simulated, or non-live input before presentation;

- visible indication of transport link, source, and data freshness;
- deterministic fallback behaviour with observable task timing and queue condition; and
- bounded, context-conditioned wellness guidance without a diagnostic or treatment claim.

The problem is constrained by the available implementation. The current genuine pathway proves documented receiver-side records and bridge eligibility rules, not a universal end-to-end guarantee for every Huawei Health screen or measurement. Physical ESP32 sensor acquisition is similarly outside the verified final runtime. Hardware selection and Wokwi artefacts provide design and simulation evidence rather than a claim of deployed sensor readings.

1.3 Limitations of Unqualified Wellness Interfaces

An interface can be technically misleading even when every displayed number originated at a legitimate device. A heartbeat value may be historical rather than real-time; a route may drop a field; a receiver may accept a packet with an old timestamp; or a dashboard may retain its last good value after the sender disconnects. KINETIQ treats arrival time, value-change time, and freshness state as first-class UI data. After the configured offline threshold, the mirror blanks the measurement rather than retaining it in a live-looking card.

The same caution applies to AI output. Context-grounded response generation can constrain an assistant to the accepted snapshot, but a displayed grounding verdict does not establish medical accuracy, clinical safety, or suitability for a particular person. KINETIQ is a wellness-assistance prototype. It does not provide medical diagnosis, treatment, emergency detection, or clinical alerting.

1.4 Objectives

The implemented objectives are organised around evidence separation and controllable operation:

1. implement an observable native FreeRTOS control lane with activity commands, task timing, queue diagnostics, and synthetic telemetry;
2. ingest Huawei Health user-interface observations through a phone-side relay and authenticated local receiver without claiming a direct Huawei Band API;
3. define and enforce a shared eight-field telemetry contract before dashboard or assistant consumption;
4. make source, link, freshness, and quality state visible to the operator;
5. generate context-conditioned wellness responses with a visible grounding `PASS`/`FAIL` verdict and offline fallback; and
6. document repeatable demonstrations, test scope, and unverified hardware boundaries.

1.5 Scope and Boundaries

The laptop application is local-only in the examined configuration: its HTTP receiver binds to the loopback interface. The phone relay sends authenticated POST requests to the local receiver. The bridge emits a strict mirror schema only when HR, SpO_2, and steps are present, valid, non-simulated, and classified as live by the relay contract. Stress observations are retained in documented receiver records but are not among the eight shared mirror fields and are not forwarded by the examined bridge.

The control lane accepts keyboard commands `1` (Rest), `2` (Walk), `3` (Run), `b` (Fall event), and `0` (Automatic mode). It is a Windows-hosted FreeRTOS simulator and cannot establish hard real-time behaviour of a deployed microcontroller. The planned embedded scope identifies an ESP32 DevKit V1, MPU6050, HC-SR04, infrared sensing, DHT22, supporting components, and a Wokwi route. Those materials are retained as design evidence only.

1.6 Dual-Pathway Overview

Figure 1.1 summarises the design principle. The genuine lane preserves provenance from the wearable application path. The deterministic lane supplies explicitly synthetic control telemetry. Both pass through a shared contract and dashboard surface, but their source labels remain separate. That separation prevents the control lane from becoming an unmarked substitute for a user measurement.

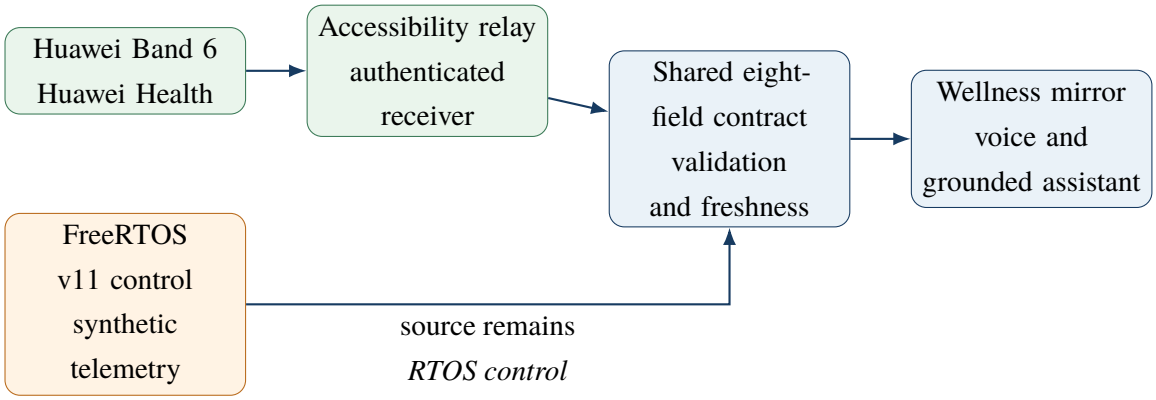


Figure 1.1: KINETIQ uses two telemetry pathways with a common validation and mirror interface. Green blocks carry genuine wearable-path observations; orange denotes synthetic control telemetry.

1.7 Chapter Organisation

The remaining chapters establish the technical background, measurable requirements, architecture, implementation details, evidence-based results, and future integration work. The sequence moves from the evidence boundary to the mechanisms that enforce it and finally to the observations that support or qualify each claim.

CHAPTER 2

LITERATURE REVIEW AND TECHNICAL BACKGROUND

2.1 Real-Time Tasks and Embedded Telemetry

An RTOS partitions work into independently scheduled tasks and provides communication primitives that preserve ownership and ordering. FreeRTOS queues copy items into an internal buffer and are commonly used to transfer messages between producer and consumer tasks [2]. Mutexes add mutual exclusion and priority-inheritance behaviour for protected shared resources [3]. These mechanisms are directly relevant to the native KINETIQ control node: simulated sensing produces samples, classification consumes them, and telemetry publishes a single coherent output stream.

Real-time terminology requires restraint. A periodic task can measure release lateness and queue accumulation on a desktop simulator, but the host operating system and console I/O prevent that observation from constituting a hard timing guarantee. KINETIQ consequently records nominal period, observed lateness, queue high-water mark, dropped events, and missed-deadline count as diagnostics rather than advertising a real-time certification.

2.2 Wearable Data and Android-Mediated Integration

Wearable ecosystems often expose data through platform-controlled applications, accounts, permissions, and health-data APIs. Huawei documents Wear Engine as a development framework for selected wearable use cases [4]; its availability and access model do not establish an implemented direct integration in KINETIQ. The examined implementation instead relies on Huawei Health's visible user-interface data and a user-enabled Android accessibility service. Android's accessibility-service documentation establishes the controlled UI-observation context and permission model [1]. It does not convert an accessibility capture into a medical-grade data interface.

This distinction leads to a useful evidence ladder. A value visible in Huawei Health is an upstream observation. A POST row at the local receiver establishes arrival at the receiver. A complete bridge output establishes eligibility for the eight-field mirror contract. A visible dashboard card establishes rendering. Each downstream claim requires its own evidence; an upstream screenshot cannot substitute for it.

2.3 Validation, Freshness, and Provenance

Network telemetry requires more than syntactically valid JSON. Field presence, range limits, timestamp interpretation, source identity, and receiver arrival time all shape whether an observation should control an interface. KINETIQ uses strict validation to reject missing or out-of-range values. Its UI then distinguishes an arriving value from an unchanged value and transitions through live, stale, and offline states. This is especially important for wellness presentation, where displaying a cached card without a status marker creates an avoidable ambiguity.

Data provenance is represented operationally rather than as an abstract label: the originating lane, simulated flag, transport class, timestamp, and link state determine whether a sample is accepted and how it is described. The current contract intentionally limits the shared mirror data to eight fields. Additional upstream observations, such as stress, remain documented at the receiver but cannot silently appear as mirror context unless the contract and forwarding logic are extended.

2.4 Local Transport and Interface Security

The Python standard library provides a basic HTTP server suitable for local development and controlled interfaces, while warning that production hardening remains the application's responsibility [5]. KINETIQ limits its examined dashboard server to `127.0.0.1` and uses an authenticated POST mechanism at the local receiver. This reduces accidental exposure on a local network but is not a complete security assessment. Secret storage, transport encryption across device boundaries, replay resistance, mobile application hardening, and independent penetration testing remain outside the documented work.

2.5 Speech and Context-Grounded Assistance

The voice pathway uses faster-whisper, an implementation associated with CTranslate2-based Whisper inference [6]. A transcription latency on a synthesized clip measures only one constrained machine-and-input condition; it does not measure live microphone robustness, accent coverage, noise performance, or medical communication safety.

Grounding in KINETIQ is narrower than retrieval-augmented generation. Retrieval-augmented generation combines a parametric generator with external non-parametric memory [7]; the examined KINETIQ assistant instead injects the accepted current telemetry snapshot into a bounded prompt or deterministic template. A local verifier checks whether key response elements agree with the accepted activity, exertion, heart-rate, steps, or PPG-quality context. The resulting verdict makes the implemented consistency check visible, but it cannot prove factual completeness or clinical validity.

2.6 Embedded Hardware Context

The ESP32 development-board documentation describes the board family and connection context used for the selected hardware [8]. Espressif also documents the importance of bootstrapping pins and schematic precautions [9]. These sources support component-aware design but do not establish that KINETIQ completed physical assembly or acquired a final sensor stream. The repository includes a Wokwi simulation route; Wokwi documents its browser-based electronics simulation environment [10]. Simulation is useful for control logic and pin-planning checks, while physical sensor noise, power integrity, enclosure effects, and wearability require later validation.

2.7 Engineering Gap

Existing RTOS guidance explains task communication, and platform documentation explains constrained wearable or Android integration. Neither alone solves the presentation problem created when genuine observations, simulated control values, stale caches, and conversational output coexist in one mirror. KINETIQ addresses that bounded gap by making source and freshness explicit, applying a contract before UI and AI use, and retaining a labelled synthetic fallback. The work does not claim that no comparable product exists; it documents a transparent integration pattern for the specific implementation and evidence available.

CHAPTER 3

OBJECTIVES AND SYSTEM REQUIREMENTS

3.1 Requirement Basis

Requirements were derived from the documented KINETIQ software, the final-review presentation, repository tests and logs, and the stated demonstration boundary. A requirement is marked verified only when the examined source or artefact supports the stated scope. A qualified status means that an implementation or a bounded test exists but does not establish every implied operational condition. Hardware planning alone is not treated as a completed requirement.

3.2 Functional Requirements

Table 3.1: Functional requirements and available verification evidence.

ID	Requirement	Implementation approach and evidence	Status
FR1	Provide controlled Rest, Walk, Run, Fall, and Automatic modes.	Native FreeRTOS console maps `1`, `2`, `3`, `b`, and `0/a` to control state.	**Verified**
FR2	Publish labelled synthetic control telemetry.	Telemetry task serialises a contract-compatible JSON payload and reports RTOS-control diagnostics.	**Verified**
FR3	Receive genuine-path Huawei Health observations locally.	Android accessibility relay and authenticated POST receiver; dated receiver CSV records are available.	**Qualified**
FR4	Accept only complete, fresh, valid genuine bridge input.	Bridge guards require HR, SpO_2, steps, non-simulated source, bounds, and live freshness.	**Verified**
FR5	Enforce one eight-field telemetry schema at the mirror.	`telemetry_schema.py` validates type, enumeration, and ranges before web ingestion.	**Verified**
FR6	Expose source and freshness to an operator.	Dashboard maintains link and value-age state: LIVE, UNCHANGED, STALE, and OFFLINE.	**Verified**
FR7	Provide sensor-conditioned wellness assistance.	Local assistant receives validated context and emits a bounded reply with a visible grounding verdict.	**Qualified**
FR8	Support voice interaction.	faster-whisper STT module and text-to-speech route exist; a synthesized-audio timing result is documented.	**Qualified**
FR9	Continue with a deterministic fallback when wearable transport is unavailable.	FreeRTOS lane remains selectable and synthetic source identity remains visible.	**Verified**

3.3 Non-Functional Requirements

The implementation gives equal importance to how a value is handled and what it contains. Table 3.2 records the operating properties relevant to a local demonstration.

Table 3.2: Non-functional requirements and design controls.

ID	Property	Design control	Status
NFR1	Freshness transparency	Arrival time, value-change time, and state thresholds drive explicit UI status; offline values are blanked.	**Verified**
NFR2	Input reliability	Strict keys, ranges, and enumerated values reject malformed packets.	**Verified**
NFR3	Observability	Native node reports period, lateness, queue HWM, drops, and missed deadlines.	**Verified**
NFR4	Locality	Dashboard binds to loopback; receiver uses a local authenticated POST contract.	**Qualified**
NFR5	Repeatability	Console commands, mock stream, BLE mock, CSV logs, and launch scripts support bounded demonstrations.	**Qualified**
NFR6	Provenance safety	Genuine and RTOS paths retain different source labels; simulated input is rejected by the genuine bridge.	**Verified**
NFR7	Medical safety	UI/assistant positioning remains wellness-only; no diagnostic or clinical-alert claim is made.	**Verified**

3.4 Shared Telemetry Contract

The browser application accepts exactly the fields in Table 3.3. The schema is intentionally narrow so that validity can be checked before the value reaches the dashboard or assistant. The Huawei bridge maps its accepted data to this contract; it does not copy arbitrary health-application fields into the mirror.

Table 3.3: Eight-field telemetry contract enforced by the shared receiver.

Field	Accepted form	Validation rule
`timestamp`	ISO-8601 string	Required; used with receiver arrival time for freshness handling.
`heart_rate`	numeric BPM	Required; 30–220.
`spo2`	numeric percent	Required; 70–100.
`activity`	string	`rest`, `walk`, `run`, `active`, or `unknown`.
`motion_intensity`	numeric	Required; 0.0–1.0.
`exertion`	string	`rest`, `light`, `moderate`, or `high`.
`steps`	integer	Required; non-negative.
`ppg_quality`	string	`good` or `poor`.

3.5 Safety and Acceptance Boundary

The accepted telemetry context is a software input, not a clinical measurement guarantee. A grounding `PASS` means the implemented verifier found required contextual agreement in the generated wording; a `FAIL` indicates that one or more checks did not agree. Neither status establishes physiological correctness, medical appropriateness, or emergency safety. Emergency, diagnosis, treatment, and medication decisions remain outside the system boundary.

3.6 Demonstration Acceptance Conditions

The minimum repeatable demonstration sequence is: start the web mirror; select either the genuine bridge or the native control stream; observe source and freshness state; submit a valid contract payload; request a bounded wellness response; and inspect the diagnostic or log evidence. A genuine-path demonstration requires a fresh, complete relay snapshot. When that condition is unavailable, the operator must identify the RTOS output as synthetic control data rather than representing it as an equivalent substitute.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 Three-Tier Architecture

KINETIQ has a sensing/control tier, a local transport and validation tier, and a laptop application tier. The first tier is deliberately dual: the Huawei route carries genuine wearable-path observations, while the native FreeRTOS route generates labelled synthetic control telemetry. The transport tier authenticates or validates input and translates eligible values to the shared contract. The application tier renders state, accepts voice or text requests, and conditions the assistant on the accepted telemetry context.

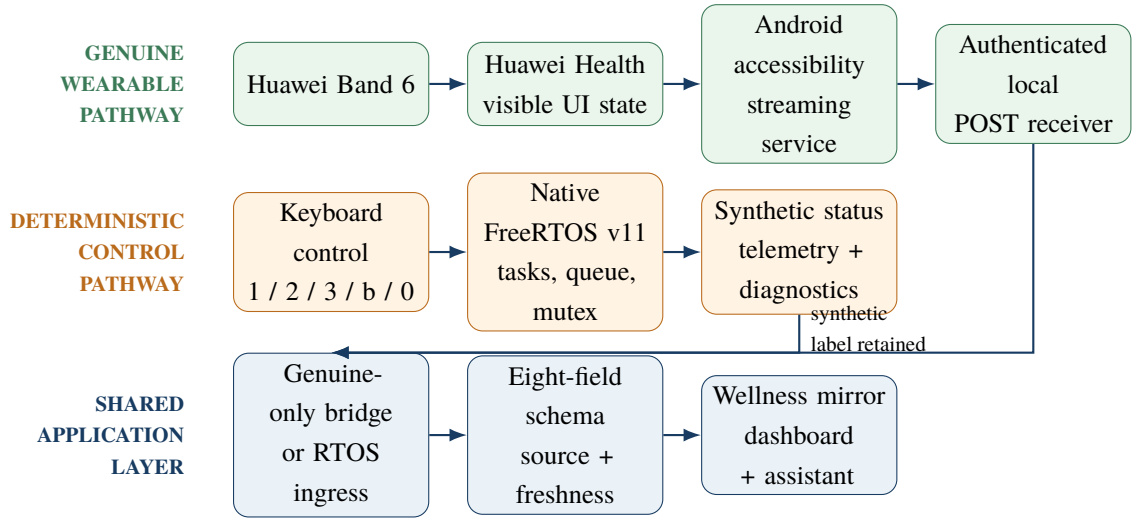


Figure 4.1: Overall KINETIQ architecture. The two source lanes converge only after their origin and eligibility are made explicit.

4.2 Genuine Wearable-Data Pathway

The genuine route starts at Huawei Band 6 observations displayed through Huawei Health. A phone-side service labelled *KinetiQ Streamer* reads the relevant UI state under Android accessibility permission and submits a local POST payload. The local receiver records the incoming source, schema version, timestamp, value modes, device model, simulation status, transport, and steps. A receiver record is evidence of arrival at that receiver; it is not by itself evidence that a later browser tab displayed the same value.

The genuine-only bridge examines the relay snapshot before writing the shared schema to the mirror process. It requires complete HR, SpO_2, and steps values; acceptable ranges; non-simulated origin; and the relay's `live` freshness classification. The bridge maps activity to `unknown`, motion intensity to `0`, PPG quality to `good` after an upstream completed SpO_2 reading, and exertion through the bridge's documented heuristic. Those mappings are compatibility values, not independently measured activity, motion, PPG signal quality, or exertion. Stress may be present in the relay record, but it is not a member of the locked eight-field mirror contract and is not forwarded by the examined bridge.

4.3 Deterministic FreeRTOS Control Pathway

The native control application is a FreeRTOS v11 Windows-simulator executable. It creates four application tasks around an eight-sample queue and a mutex-protected shared control state. The console task reads operator commands approximately every 50 ms. The sensor task produces virtual IMU and DHT-like input at 100 ms intervals. The classification task drains samples and applies activity hysteresis at a nominal 1 Hz. The higher-priority telemetry task publishes one synthetic JSON status at a nominal 1 Hz and is the sole standard-output writer.

Table 4.1: Native FreeRTOS task structure in the examined implementation.

Task	**Nominal cadence**	**Priority**	**Role**
Console	about 50 ms poll	idle + 1	Reads rest, walk, run, fall, and automatic commands; updates protected control state.
Sensor	100 ms	2	Generates virtual IMU/DHT-like samples; sends them to an eight-slot queue.
Classify	1 s	2	Drains queue and applies state/hysteresis logic.
Telemetry	1 s	3	Builds synthetic contract telemetry; reports timing, queue HWM, drops, and missed deadlines.

The fall command is a control event in this simulator. It must not be interpreted as a validated fall-detection algorithm or a physical sensor result. The queue is configured to retain the newest virtual inputs when pressure occurs by dropping the oldest entry, with high-water mark and dropped-event counters exposed in diagnostics.

4.4 Telemetry Validation and Freshness Workflow

Figure 4.2 depicts the shared decision flow. Syntax alone is insufficient. Authentication applies to the receiver route; schema, complete-field, range, source, and freshness checks determine bridge or dashboard acceptance. A valid value that ceases to arrive is not kept in a live state indefinitely. The web layer tracks receiver arrival and fingerprint changes and transitions between `LIVE`, `UNCHANGED`, `STALE`, and `OFFLINE`; the documented values are blanked once offline.

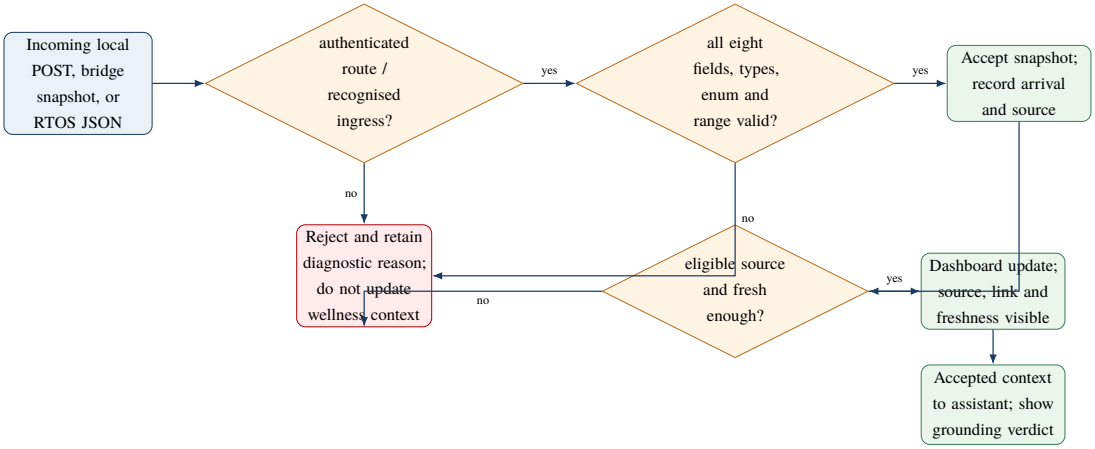


Figure 4.2: Validation and freshness workflow. The genuine bridge adds source-specific completeness and live-freshness guards before it emits the shared contract.

4.5 Dashboard and Assistance Layer

The laptop dashboard runs a threaded HTTP server on loopback, tries port 8777 and then 8778 when necessary, and ingests valid JSON through standard input or application endpoints. It maintains a snapshot plus arrival and value-change times. The cards show accepted wellness context along with the source and data state. Browser-camera handling is limited to a pixel-difference motion proxy; no implemented pose or posture classifier is claimed.

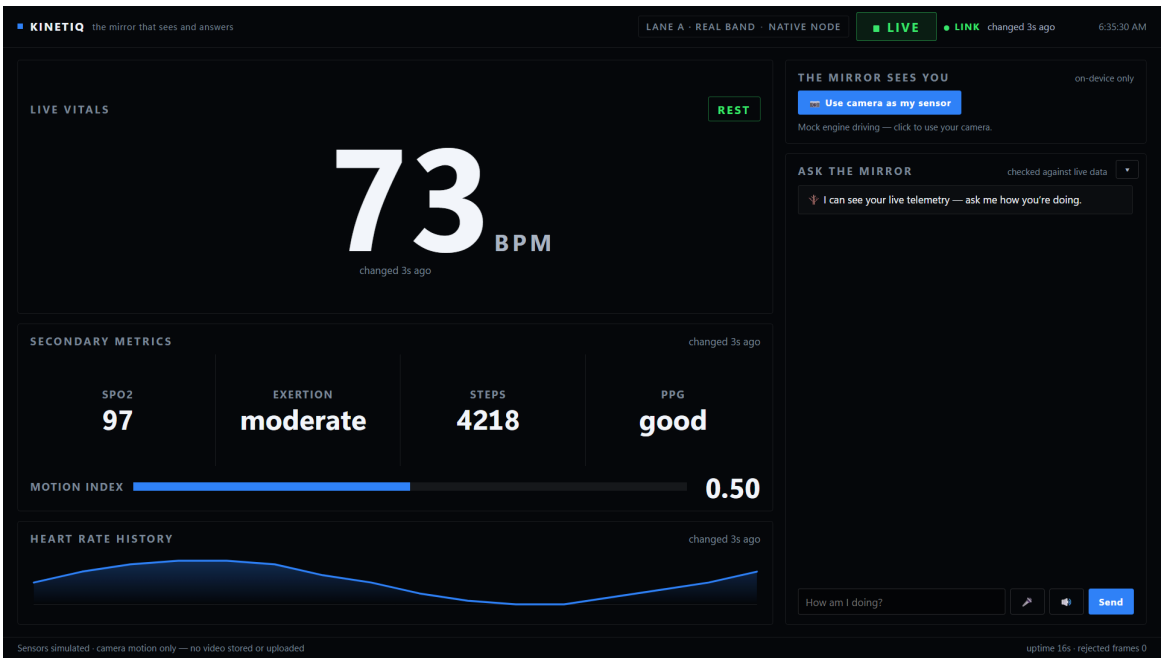


Figure 4.3: Implemented dashboard mirror rendered in the browser during a controlled local input session; the displayed stream is synthetic, not live wearable telemetry.

The assistant accepts the current validated context, attempts a local Ollama `gemma2:2b` response where available, and otherwise uses a deterministic template.

Responses are bounded to 70 words.

The verifier checks specified agreement with activity, exertion, an exact HR/heart value or step count, and poor-quality wording when applicable. The process shows `PASS` or `FAIL`; the examined implementation still sends text to speech after a failed check, so the verdict is informative rather than a fail-closed safety gate.

4.6 Hardware Design Scope

The selected embedded components are an ESP32 DevKit V1, MPU6050, HC-SR04, infrared sensor, DHT22, and supporting wiring/prototyping materials. The materials include pin mapping and a Wokwi sketch. Figure 4.4 reproduces the project material's earlier hardware-path diagram as a conceptual design reference. It is not used as evidence of final physical sensor acquisition.

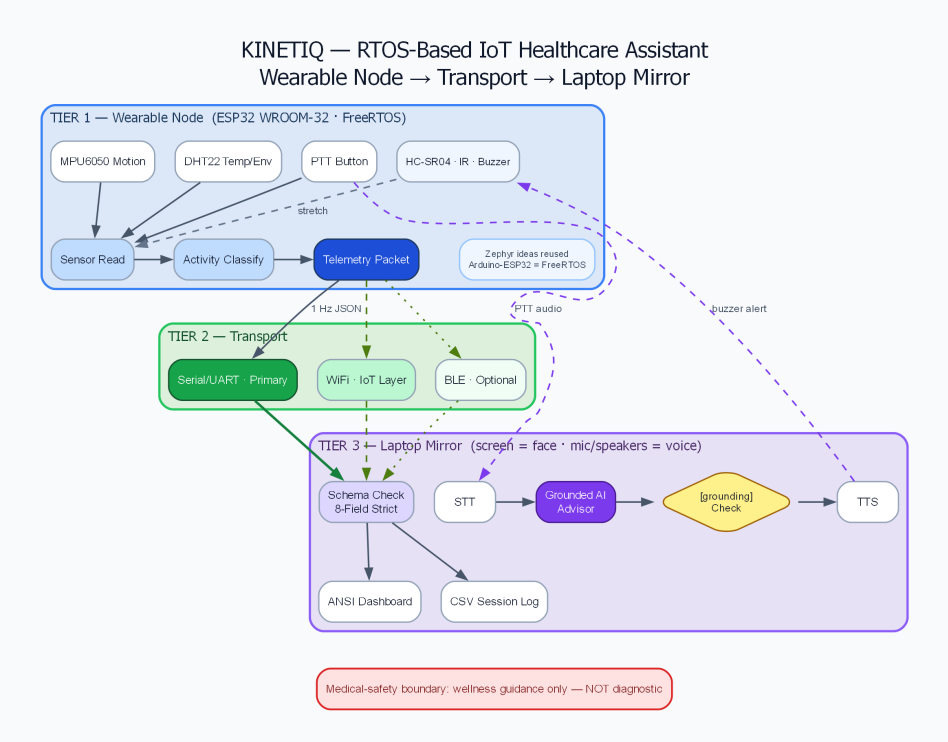


Figure 4.4: Earlier hardware-path conceptual architecture from the KINETIQ project materials. The physical ESP32 sensing path remained planned/simulated at the documented final demonstration.

The ESP32 and sensor selection remains technically appropriate for a future sensing node, but physical wiring, calibration, power behaviour, environmental response, and end-to-end sensor ingestion require separate validation. Wokwi execution can exercise the modelled sketch, not replace those measurements.

CHAPTER 5

IMPLEMENTATION AND METHODOLOGY

5.1 Software Organisation

The implementation is organised as cooperating, deliberately narrow modules. The native FreeRTOS source at `hardware/native/kinetiq_tasks.c` produces synthetic control messages and diagnostics. The Python receiver package contains the shared schema, BLE mock, bridge-related helpers, and local data handling.

The web application at `laptop/web/app.py` owns dashboard state and exposes local endpoints. The AI and voice modules reside under `laptop/ai`. This separation keeps a transport adapter from becoming an unexamined source of truth for the UI.

Table 5.1: Principal implementation modules and responsibilities.

Module	Responsibility	Evidence category
`kinetiq_tasks.c`	FreeRTOS task creation, synthetic samples, activity state, queue/mutex use, timing and JSON telemetry.	Implemented native control runtime
`telemetry_schema.py`	Required-key, type, range, and enumeration validation for common payloads.	Implemented shared validation
`band_bridge.py`	Maps eligible relay snapshots to the locked mirror schema; blocks simulated, incomplete, non-live, and out-of-range input.	Implemented bridge logic
`app.py`	Loopback dashboard server, state ageing, valid JSON ingestion, API routes, session context.	Implemented local application
`assistant.py`	Context prompt/template, response word limit, grounding checker, TTS invocation.	Implemented assistant logic
`voice_pipeline.py`	Local faster-whisper transcription path and audio compatibility handling.	Implemented voice module
`hardware/sim`	Wokwi sketch and supporting simulation configuration.	Simulation/design evidence

5.2 FreeRTOS Tasks, Queue, and Synchronisation

The control node initialises an eight-element sample queue and a mutex for the shared control state, then creates console, sensor, classification, and telemetry tasks. The sensor task produces virtual sensor samples at 100 ms. The classification task consumes queued samples once per second and uses hysteresis to reduce rapid state changes. A higher-priority telemetry task publishes a one-second status payload. Standard output ownership is confined to the telemetry task, which prevents interleaved JSON fragments from multiple writers.

Timing diagnostics use a monotonic high-resolution counter on the host. For each periodic release the code records intended period, measured period, maximum lateness, queue high-water mark, dropped events, and missed deadlines. A documented lateness threshold of 120 ms contributes to the missed-deadline diagnostic. These figures aid repeatable debugging of the desktop simulator; they do not establish a scheduling bound on every Windows execution.

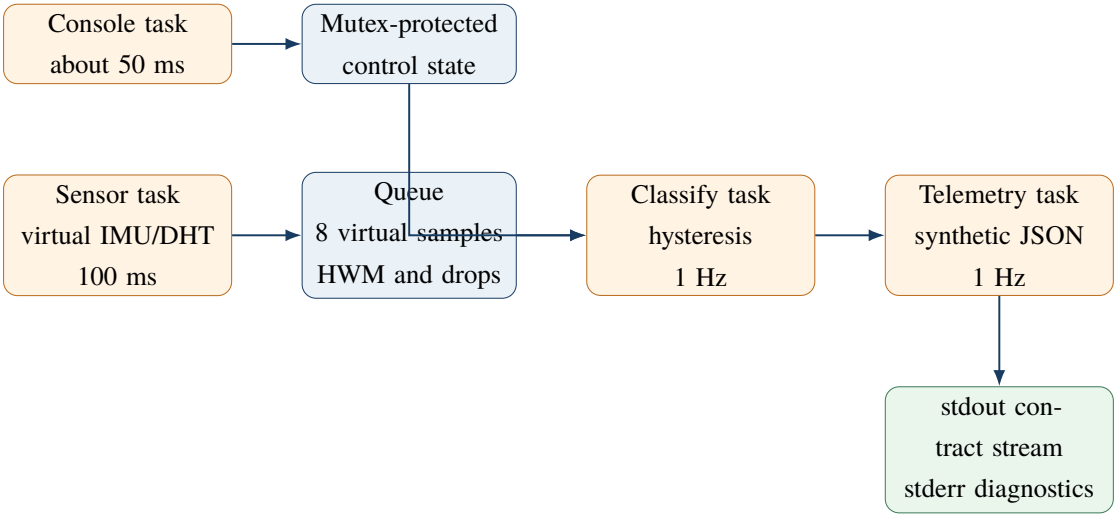


Figure 5.1: Native FreeRTOS control data path. All values emitted here are synthetic RTOS-control telemetry.

5.3 Genuine-Path Ingestion and Bridging

The Android relay converts relevant Huawei Health interface observations into a local payload. The authenticated receiver persists each accepted relay row with source and transport metadata. The bridge then uses an all-or-nothing policy for the mirror's required physiological fields: HR, SpO_2, and steps must all exist and pass range checks. Relay freshness must be `live`, and `simulated` must be false. Any failed precondition blocks forwarding.

The following pseudocode captures the bridge decision without exposing application secrets or unrelated implementation detail:

Listing 5.1: Genuine-only bridge policy, simplified.

```
snapshot = read_latest_authenticated_relay_snapshot()
if snapshot.simulated or snapshot.freshness != "live": reject()
if missing(snapshot.hr, snapshot.spo2, snapshot.steps): reject()
if not in_range(snapshot.hr, 30, 220): reject()
if not in_range(snapshot.spo2, 70, 100): reject()
if snapshot.steps < 0: reject()

emit_to_mirror({timestamp, heart_rate, spo2, activity="unknown",
  motion_intensity=0, exertion=bridge_heuristic(...),
  steps, ppg_quality="good"})
```

The PPG-quality assignment is an upstream-completion proxy, not a signal-quality algorithm. The exertion field is a bridge heuristic. The reporting language therefore treats the bridge output as validated transport context with selected derived fields, not as a fully measured physiological model.

5.4 Schema Validation and Dashboard State

The web ingress parses JSON, validates the eight required keys, and ignores non-JSON standard-input lines.

The validator restricts activity to `rest`, `walk`, `run`, `active`, or `unknown`; exertion to `rest`, `light`, `moderate`, or `high`; and PPG quality to `good` or `poor`. Numeric boundaries are listed in Table 3.3. Rejected packets do not replace the accepted wellness snapshot.

Freshness is implemented in the web layer with a 60 s live interval, a subsequent 60 s stale interval, and an offline limit at 300 s. The code additionally detects whether a newly arriving packet changed the snapshot fingerprint. This distinguishes a recently arriving repeated value (`UNCHANGED`) from a value that has stopped arriving (`STALE` or `OFFLINE`). The value display is blanked in the offline condition.

Table 5.2: Dashboard freshness and link-state treatment.

State	Condition in examined UI logic	Interface consequence
LIVE	Recent arrival with a changed fingerprint inside the live interval.	Current card and source state are displayed.
UNCHANGED	Recent arrival but repeated fingerprint.	Card remains visible; repetition is not disguised as a changed sample.
STALE	Arrival age exceeds the live interval but remains below offline threshold.	Stale status is visible.
OFFLINE	Arrival age exceeds 300 s.	Documented interface blanks measurement values and shows offline state.

5.5 Grounded Assistance and Voice Route

The assistant constructs a local context containing accepted telemetry fields and generates through an optional local Ollama `gemma2:2b` endpoint. When that endpoint is unavailable, a deterministic template remains available. The response is limited to 70 words. The verifier inspects required contextual mentions: activity and exertion normally appear in the response; an exact heart-rate/heart wording or exact step count relaxes the requirement for both; and poor PPG quality requires qualifying language. The result is surfaced as `PASS` or `FAIL` in the application.

Speech input is transcribed through faster-whisper `tiny.en` with `int8` inference. Text-to-speech is invoked for the generated answer. A synthetic-clip result measures the transcription route in a controlled condition only. Browser camera input, where used, supplies a motion proxy based on frame-pixel differences and is not a body-pose system.

5.6 Launch and Fallback Procedure

The reproducible operator route is deliberately simple:

1. start the local web application and confirm its loopback port;
2. start the authenticated receiver and either the genuine-only bridge or the native FreeRTOS control executable;
3. inspect the dashboard source and freshness card before discussing a value;

4. use a fresh complete Huawei relay snapshot only when a genuine-path demonstration is required;
5. select the RTOS lane when transport is unavailable, retaining its synthetic label; and
6. retain console diagnostics and CSV/session artefacts for the demonstration record.

The fallback retains operability, not equivalence: RTOS synthetic values cannot substitute for unavailable wearable observations.

5.7 Hardware Simulation and Integration Status

The Wokwi sketch models the embedded direction with task-oriented code and simulated inputs. It supports early pin and logic exploration. The selected physical BOM and pin map support planned assembly, but the final documented demonstration does not claim newly acquired ESP32 readings. No section of the runtime validation should be interpreted as a completed physical MPU6050, HC-SR04, infrared, or DHT22 integration.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Evidence Classification

The results are classified by pathway and test condition before interpretation. A receiver CSV row, a mock stream, a simulator diagnostic, and a physical sensor trace have different evidential meaning. Table 6.1 prevents a successful bounded test from being presented as a broader system claim.

Table 6.1: Evidence classification used for reported observations.

Evidence	What it establishes	What it does not establish
Huawei relay receiver row	A relay payload reached and was stored by the authenticated local receiver.	Browser display of every row; clinical validity; direct Huawei API use.
Dashboard mock CSV	A synthetic stream was processed under its recorded run condition.	Live wearable throughput or clinical telemetry performance.
BLE mock counter	Mock routing handled the stated message count.	Device-specific BLE pairing or Huawei Band transport.
FreeRTOS host diagnostics	Behaviour of the Windows-hosted control runtime.	Hard real-time guarantee or genuine physiology.
Wokwi sketch	Simulated hardware/control logic and planned pin pathway.	Physical sensor acquisition or calibration.

6.2 Genuine Wearable-Path Observation

The receiver log contains Huawei-Health relay entries dated 22 October 2026. One complete record stores heart rate 60 BPM, SpO_2 100%, stress 57, and 4,181 steps, identified as `huawei_band_6_healthapp`, `simulated=false`, and transport `health_app_ui`. The same record classifies heart rate and stress as real-time while labelling SpO_2 historical. A nearby earlier record stores 74 BPM, 100% SpO_2, stress 57, and 4,181 steps. These are documented relay observations, not clinical measurements.

Table 6.2: Selected documented Huawei-Health relay observations at the local receiver.

Receiver time	HR	SpO_2	Stress	Steps	Qualification
2026-10-22 13:14:35	74 BPM	100%	57	4,181	Complete relay record; health-app UI transport.
2026-10-22 13:17:37	60 BPM	100%	57	4,181	HR/stress marked real-time; SpO_2 marked historical in the stored relay metadata.
2026-10-23 01:58	78 BPM	99%	absent	5,432	Later complete HR/SpO_2/steps record; stress absent.

The bridge accepts only fresh, complete HR, SpO_2, and steps according to its own contract. Because stress is not a shared schema field, the recorded stress observation cannot be claimed as mirror context. The inspected materials establish receiver-side acceptance and bridge eligibility logic; they do not contain a corresponding browser capture proving that every selected row reached the mirror. The genuine result is therefore qualified as **Qualified**.

6.3 Schema and Bridge Validation

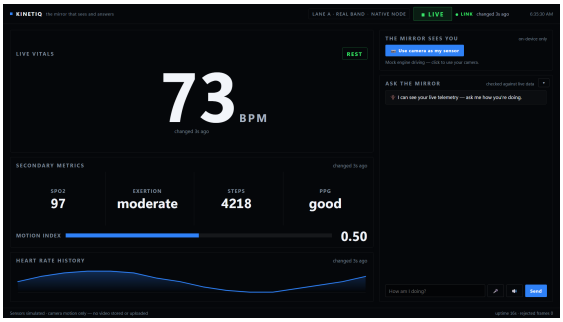
Integration logic covers complete live snapshots, missing HR/SpO_2/steps, stale values, simulated origin, and out-of-range values. The expected acceptance mapping for a valid live snapshot includes HR 74, SpO_2 100, and steps 4,181 with activity `unknown`, motion intensity zero, `good` PPG quality proxy, and heuristic exertion. The negative paths demonstrate an important safety property: malformed or ineligible genuine-lane input is blocked before it becomes an apparently current dashboard value.

No statistic for live rejection rate is reported because a controlled representative sample count and test log for that rate were not identified. The result establishes branch behaviour in the available integration test and source, not a field reliability percentage.

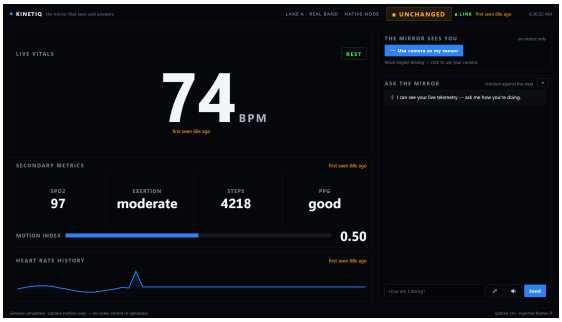
6.4 Dashboard Mock Stream

The recorded dashboard session `session_20261014_193826.csv` contains 75 telemetry frames over 15 seconds with zero dropped frames in the associated documented mock run. Figure 6.2 retains the source chart's explicit synthetic label. The chart consists of a mock

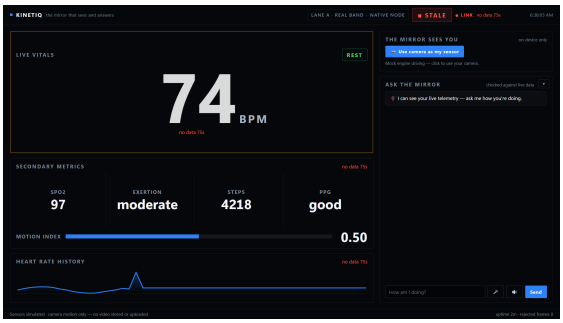
sensor stream, so its apparent time trends must not be interpreted as genuine wearable biometrics.



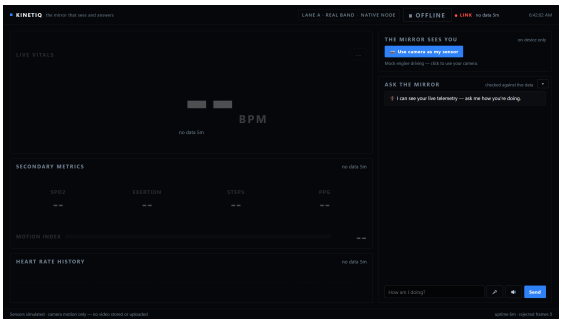
(a) Live



(b) Unchanged



(c) Stale



(d) Offline

Figure 6.1: Dashboard freshness states observed under controlled synthetic input: live, unchanged, stale and offline. These captures exercise the freshness/offline logic; they are not live wearable telemetry.

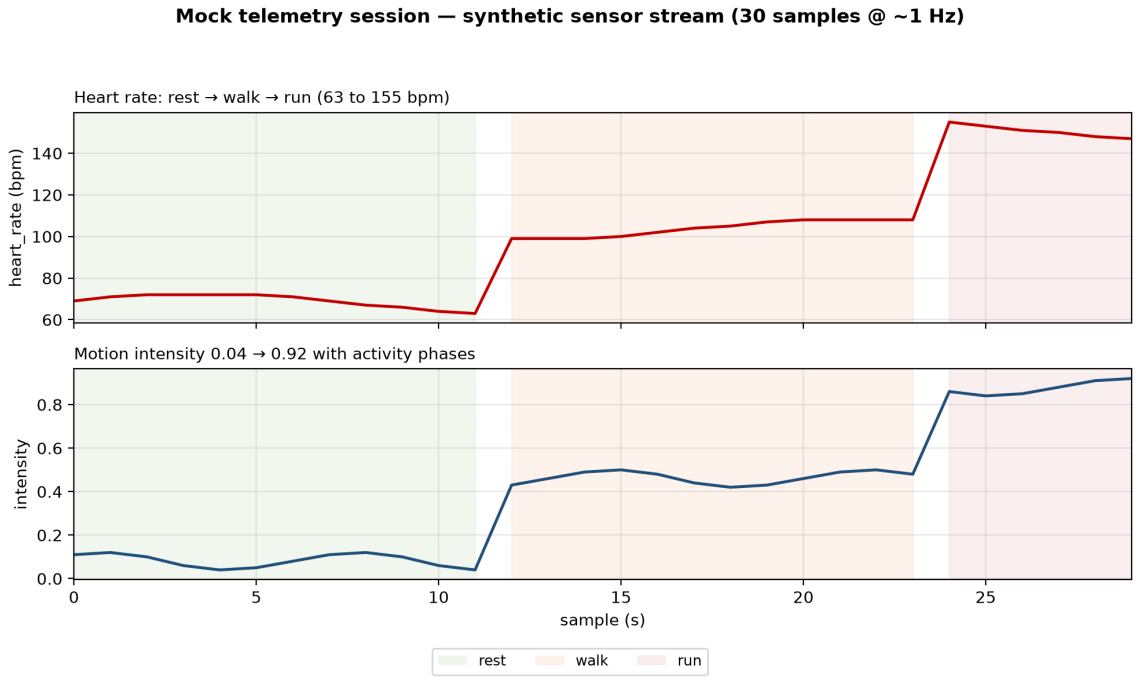


Figure 6.2: Mock telemetry session: synthetic sensor stream (30 samples at approximately 1 Hz) from the project materials. It is not a genuine biometric time series.

The zero-drop result establishes that the documented mock dashboard session maintained its recorded frame count under those input and host conditions. It does not establish throughput for the phone relay, an arbitrary browser load, or all live wearable sessions.

6.5 BLE Mock and Voice Observation

The BLE client mock is configured with `--source mock --mock-count 30`; the project result records 30 routed messages with zero rejections. This confirms the mock route and its message handling for the defined input. It does not confirm physical BLE communication with Huawei Band 6 or any other wearable.

The voice pipeline result records 0.48 s for faster-whisper transcription of a synthesized audio clip. The software uses the `tiny.en` model with `int8` inference. The metric is useful for a local demonstration latency budget, but it does not assess live microphone capture, noise, language variation, or correctness of a wellness utterance.

6.6 RTOS Control and Offline Fallback

The native control node exposes period, lateness, queue high-water mark, dropped events, and missed-deadline fields while generating contract-compatible synthetic telemetry. Rest, walk, run, fall, and automatic controls are mapped through the console. The single-writer design and diagnostic fields make the fallback observable and reduce ambiguity during a demonstration. The host execution remains a Windows simulator, so the results support source-level task design and runtime diagnostics rather than a hard-deadline assertion.

When genuine transport cannot provide a complete live snapshot, the RTOS route supports a repeatable interface demonstration. It must be labelled as control or simulated data in the dashboard and narration. Treating the synthetic lane as a physiological backup would defeat the provenance boundary that the design is intended to protect.

6.7 Grounding Verdict Behaviour

The assistant renders a grounding verdict after comparing response text with accepted telemetry. A `PASS` denotes agreement under the implemented keyword/value checks. A `FAIL` identifies a mismatch in those checks. Neither observation validates medical content. The available code continues text-to-speech after the verdict, including a failure, so a `FAIL` is a visible warning rather than a suppression mechanism. Further work should make output gating and test fixtures explicit if fail-closed behaviour is required.

6.8 Hardware and Budget Reconciliation

The physical embedded hardware remains planned or simulated. The final-review component table lists INR 998 for two ESP32 boards, INR 470 for the MPU6050/HC-SR04/IR group, and INR 808 for DHT22 plus breadboard/jumper support. Those listed subtotals sum to INR 2,276. The same source also contains an approximate INR 1,950 Tier-A statement, which is not supported by the displayed component amounts. Table 6.3 retains the reproducible subtotal and excludes the inconsistent figure from any verified budget claim.

Table 6.3: Reconciled component subtotal from the displayed final-review BOM.

Displayed category	**Amount**	**Interpretation**
Two ESP32 development boards	INR 998	Listed category subtotal.
MPU6050, HC-SR04, and IR group	INR 470	Listed category subtotal.
DHT22, breadboard, and jumper group	INR 808	Listed category subtotal.
Recalculated displayed subtotal	**INR 2,276**	998 + 470 + 808.
Earlier Tier-A statement	INR 1,950	Not retained as verified: its revision/itemisation is not recoverable from the displayed table.

6.9 Overall Verification Summary

The strongest verified elements are the eight-field validation logic, dashboard freshness design, native FreeRTOS task structure, synthetic fallback, and documented local receiver records. The genuine wearable path is partially demonstrated through relay data and bridge rules but lacks complete browser-render evidence for each cited observation. Voice and grounding modules are implemented but only receive narrow test evidence. Physical sensors remain outside the validated final runtime. These qualifications preserve the engineering value of the working modules without extending the evidence beyond its scope.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

KINETIQ establishes a transparent local wellness-mirror architecture built around two non-equivalent telemetry pathways.

The Huawei Band 6 route carries Huawei Health user-interface observations through an Android accessibility relay and authenticated local receiver. The native FreeRTOS v11 route supplies deterministic, explicitly synthetic control telemetry with queue and timing diagnostics. The common application layer accepts only an eight-field schema, exposes source and freshness state, and conditions assistance on the accepted context.

The documented evidence supports receiver-side Huawei relay records, bridge eligibility logic, schema enforcement, dashboard mock streaming, BLE mock routing, a synthesized-audio transcription observation, and the FreeRTOS control design. It does not support a direct Huawei Band API claim, clinical validity, a completed physical ESP32 sensing node, or a claim that every documented receiver record reached the browser mirror. The report preserves those distinctions. The grounding verdict offers visible consistency feedback under a limited rule set; it is not a medical-safety certification and is not presently fail-closed.

The central engineering contribution is the provenance and freshness boundary: the interface distinguishes genuine wearable-path data from RTOS control data and stops presenting offline values as current. That behaviour is more defensible for a wellness demonstration than an interface that merges valid-looking but differently sourced values without qualification.

7.2 Future Scope

Priority future work follows directly from the outstanding evidence gaps:

1. complete a physical ESP32 sensor node and record calibrated MPU6050, HC-SR04, infrared, and DHT22 acquisition under stated test conditions;
2. create end-to-end trace fixtures that link a Huawei Health UI capture, authenticated receiver row, bridge emission, and dashboard rendering without exposing personal data;
3. extend the shared contract only through versioned validation, including an explicit decision on stress representation and its freshness semantics;
4. replace proxy mappings for motion, PPG quality, and exertion with independently measured or clearly qualified sources;

5. test receiver authentication, secret handling, replay behaviour, local-network exposure, and error recovery beyond the present loopback demonstration;
6. add automated tests for stale-to-offline blanking, assistant grounding failure, and output-gating policy; and
7. evaluate speech interaction with consented live audio, realistic acoustic conditions, and safety-reviewed wellness wording.

These steps would strengthen integration confidence while retaining the current design principle: no value, label, or assistant response should imply a level of physiological or clinical certainty that the underlying evidence does not support.

CHAPTER A

SUPPORTING TECHNICAL INFORMATION

A.1 Operator Command Map

Table A.1: Native FreeRTOS control commands.

Key	Control state	Interpretation
`1`	Rest	Synthetic low-activity control state.
`2`	Walk	Synthetic walking control state.
`3`	Run	Synthetic running control state.
`b`	Fall event	Simulator control event; not a validated physical fall detection.
`0` or `a`	Automatic mode	Returns the virtual control logic to automatic behaviour.

A.2 Selected Receiver Metadata

The genuine relay receiver retains metadata beyond the shared mirror contract: `received_at`, `source`, `schema_version`, `heart_rate_mode`, `spo2_mode`, `stress_mode`, `status`, `device_model`, `simulated`, and `transport`.

This metadata supports later audit of the difference between an upstream Huawei Health observation and a bridge-eligible mirror packet. It is not all displayed to the mirror.

Reproduction and administrative notes. The accompanying `README.md` lists the source layout and XeLaTeX/Biber compilation sequence. The logo and project visuals required by the document are stored within this folder; original project inputs remain outside it. Register numbers, academic-year entry, certificate approval date, signatures other than the named

supervisor, and official signatures remain deliberately marked for authorised institutional completion.

REFERENCES

[1] Android Developers. *AccessibilityService*. 2026. URL: https://developer.android.com/reference/android/accessibilityservice/AccessibilityService (visited on 10/23/2026).

[2] Amazon Web Services. *FreeRTOS Kernel: Queues, Mutexes and Semaphores — Queues*. 2026. URL: https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/02-Queues-mutexes-and-semaphores/01-Queues (visited on 10/23/2026).

[3] Amazon Web Services. *FreeRTOS Mutexes*. 2026. URL: https://freertos.org/Real-time-embedded-RTOS-mutexes.html (visited on 10/23/2026).

[4] Huawei Developers. *HUAWEI Wear Engine*. 2026. URL: https://developer.huawei.com/consumer/en/hms/huawei-wearengine/ (visited on 10/23/2026).

[5] Python Software Foundation. *http.server — HTTP servers*. 2026. URL: https://docs.python.org/3/library/http.server.html (visited on 10/23/2026).

[6] SYSTRAN. *faster-whisper: Faster Whisper transcription with CTranslate2*. 2026. URL: https://github.com/SYSTRAN/faster-whisper (visited on 10/23/2026).

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K"uttler, Mike Lewis, Wen-tau Yih, Tim Rockt"aschel, Sebastian Riedel, and Douwe Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[8] Espressif Systems. *ESP32-DevKitC V4 Getting Started Guide*. 2026. URL: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user_guide.html (visited on 10/23/2026).

[9] Espressif Systems. *ESP32 Hardware Design Guidelines: Schematic Checklist*. 2026. URL: https://docs.espressif.com/projects/esp-hardware-design-guidelines/en/latest/esp32/schematic-checklist.html (visited on 10/23/2026).

[10] Wokwi. *Wokwi Documentation*. 2026. URL: https://docs.wokwi.com/ (visited on 10/23/2026).